# 2° Parcial – Programación III

## Ampliación del Sistema de Gestión de Turnos Médicos

---


- **Nombre y apellido:** Gastón Govino del Jacques
- **Carrera:** Técnico Superior en Desarrollo de Software
- **Institución:** CENT 35 "Prof. Julián Godoy"
- **Profesora:** Silvana Páez Jiménez

- **Materia:** Programación III
- **Fecha de entrega:** 13 de noviembre de 2026
- **Repositorio GitHub:** https://github.com/dev-gastong/programacion-3-parcial-2c.git

---

# Validaciones incorporadas

Se incorporaron validaciones en el archivo

```
JavaScript
backend/validators/turnoValidator.js
```

aplicadas mediante el middleware

```
JavaScript
backend/middlewares/validarEstadoTurno.js
```

sobre la ruta POST

```
JavaScript
/api/turnos
```

## Validación 1: Fecha y hora no anteriores a la actual

**Archivo**:

```
JavaScript
backend/validators/turnoValidator.js
```

**Función**:

```
JavaScript
validarFechas()
```

Verifica que la fecha del turno no sea anterior a la fecha actual. Además, si el turno es para el mismo día, controla que la hora no sea anterior a la hora actual del sistema.

**Regla**:

```
None
fecha_turno >= hoy
    si fecha_turno == hoy
        → hora_turno >= hora actual
```

El error se devuelve al frontend como JSON y se muestra al usuario en el formulario.

## Validación 2: Médico sin dos turnos en el mismo horario

**Archivo:**

```
JavaScript
backend/validators/turnoValidator.js
```

**Función**:

```
JavaScript
validarMedico()
```

**Modelo:**

```
JavaScript
backend/models/turnoModel.js
```

**Función**:

```
JavaScript
existeMedico()
```

Consulta la base de datos para verificar que el médico asignado no tenga otro turno registrado en la misma fecha y hora. Si ya existe, se rechaza la solicitud.

**Regla**:

```
None
No puede existir otro turno con el mismo
medico_confirmacion, fecha_turno y hora_turno.

Adicionalmente se valida que el equipo tampoco tenga dos turnos en el mismo horario:

(validarEquipo()), y que los datos obligatorios estén completos
(validarNulos(), validarDNI()).
```

---

# Gestión de perfiles, sesiones y mejora funcional

## Gestión de perfiles y sesiones

El sistema cuenta con tres perfiles: administrador (rol 1), médico (rol 2) y paciente (rol 3).

### Login y generación de token

**Archivo:**

```
JavaScript
backend/controllers/authController.js
```

**Función:**

```
JavaScript
login()
```

Al iniciar sesión, se verifican las credenciales contra la base de datos. Si son correctas, se genera un token JWT que incluye id_usuario, email e id_rol. El token tiene una validez de 1 hora.

### Protección de rutas en el backend

**Archivos:**

```
JavaScript

// Ruta:
backend/middlewares/authMiddleware.js

// valida que el token esté presente y sea válido.
→ verificarToken()
```

```
JavaScript
// Ruta:
backend/middlewares/autorizarRol.js

// verifica que el rol del usuario coincida
        con el requerido por la ruta.
→ autorizarRol(...roles)
```

```
JavaScript
// aplica ambos middlewares en cada ruta según el perfil habilitado.

// Ruta:
backend/routes/router.js
```

```
JavaScript
// METODOS HTTP:


// → autorizarRol(1) solo admin.
GET /api/turnos

// → autorizarRol(1) solo admin.
GET /api/turnos/:id

// → autorizarRol(1) solo admin.
POST /api/turnos

// → autorizarRol(3) solo paciente.
PUT /api/turnos/:id/cancelar
```

Un usuario que intente acceder a una ruta sin el rol correcto recibe un error 403 Forbidden.

---

### Adaptación del frontend por rol

**Archivos:**

```
JavaScript
frontend/js/app.js

// al hacer login guarda en sessionStorage
                el token, el rol y el email.
```

JavaScript

frontend/pages/dashboard.html

```
// contiene tres secciones
        (#seccion-admin, #seccion-medico, #seccion-paciente)
                ocultas por defecto.
```

JavaScript

frontend/js/dashboard.js

```
// lee el rol del sessionStorage y muestra solo la sección correspondiente, además de saludar al usuario con su email y rol
```

| **Rol** | **Sección visible** | **Funcionalidad** |
|---|---|---|
| Admin (1) | #seccion-admin | Listado completo de turnos |
| Médico (2) | #seccion-medico | Información de sus turnos |
| Paciente (3) | #seccion-paciente | Formulario nuevo turno + cancelar turno |

Si el usuario intenta acceder al dashboard sin token, es redirigido automáticamente al login.

---

## Mejora funcional — Cancelación de turnos

Se implementó la posibilidad de que un paciente cancele un turno existente, con integración completa entre frontend, backend y base de datos.

### Frontend

**Archivo:**

```JavaScript
frontend/js/dashboard.js

// Función
iniciarPaciente()
```

La sección del paciente incluye un formulario donde se ingresa el número de turno a cancelar.

Al enviarlo, el frontend realiza una petición PUT /api/turnos/:id/cancelar con el token en el encabezado Authorization.

**Backend — Ruta**

```JavaScript
// Archivo
backend/routes/router.js

// PUT
/api/turnos/:id/cancelar

// Funciones
verificarToken → autorizarRol(3) → cancelarTurno
```

**Backend — Controlador**

```JavaScript
// Archivo
backend/controllers/turnoController.js

// Funcion
función cancelarTurno()
```

Recibe el id del turno por parámetro de URL, llama al servicio y devuelve el resultado. Si hay un error de negocio (turno inexistente o ya cancelado), responde con 400. Si hay un error del servidor, responde con 500.

**Backend — Servicio y base de datos**

```JavaScript
// Archivo
backend/services/turnoService.js

// Funcion
cancelarTurno(id)
```

Consulta la base de datos para verificar que el turno exista. Verifica que el turno no esté ya en estado 'Cancelado' (validación de negocio). Si pasa ambas verificaciones, ejecuta el UPDATE en la tabla turno.

```SQL
    UPDATE turno SET estado_turno = 'Cancelado' WHERE id_turno = ?
```

El resultado se refleja inmediatamente en la base de datos y el frontend muestra el mensaje de confirmación o error al usuario.

## Archivos modificados

| **Archivo** | **Cambio realizado** |
|---|---|
| frontend/js/app.js | Guardar rol y email en sessionStorage al hacer login |
| frontend/pages/dashboard.html | Tres secciones por rol (admin, médico, paciente) |
| frontend/js/dashboard.js | Lógica de roles, saludo personalizado, formularios de turno y cancelación |
| backend/routes/router.js | Rutas diferenciadas por rol + nueva ruta PUT /:id/cancelar |
| backend/controllers/turnoController.js | Nueva función cancelarTurno |
| backend/services/turnoService.js | Nueva función cancelarTurno con validación y UPDATE a MySQL |

| | |
|---|---|
| backend/validators/turnoValidator.js | Validaciones de fecha/hora y conflicto de médico (ya existentes, revisadas) |